```
# S.T.A.R. Model for predicting log_SFR_Ha using the S.T.A.R. framework
                                ^(Star-Formation-Rate)^

# STAR: Symbolic. Tuned. Astronomical Regressors

# - S: Symbolic-Regressor (via gplearn)
# - T: Tuned (hyperparameter optimization with Optuna)
# - A: Astronomical (predicting star formation rates in galaxies)
# - R: Regressors (Random Forest and Gradient Boosting)

import pandas as pd
import numpy as np
import shap
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from gplearn.genetic import SymbolicRegressor
import optuna
import warnings
from numpy.polynomial import Polynomial
from scipy.spatial import cKDTree

warnings.filterwarnings("ignore")

# Load the original dataset (Training dataset)
df = pd.read_csv("merged_data.csv")

# Load the SDSS datasets and merge them (swap datasets as needed)
sdss_dr16 = pd.read_csv("V154sdss16.csv")  # V/154: SDSS DR16
sdss_dr12 = pd.read_csv("V147sdss12.csv")  # V/147: SDSS DR12

# Merge V/154 and V/147 on objID
sdss_merged = pd.merge(
    sdss_dr16[['objID', 'RA_ICRS', 'DE_ICRS', 'zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag',
'e_umag', 'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']],
    sdss_dr12[['objID', 'RA_ICRS', 'DE_ICRS', 'zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag',
'e_umag', 'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']],
    on='objID',
    how='outer',
    suffixes=('_dr16', '_dr12')
)
```

```
# Combine columns, preferring DR16 values where available, otherwise use DR12
sdss_merged['RA_ICRS'] =
sdss_merged['RA_ICRS_dr16'].combine_first(sdss_merged['RA_ICRS_dr12'])
sdss_merged['DE_ICRS'] =
sdss_merged['DE_ICRS_dr16'].combine_first(sdss_merged['DE_ICRS_dr12'])
sdss_merged['zsp'] = sdss_merged['zsp_dr16'].combine_first(sdss_merged['zsp_dr12'])
sdss_merged['umag'] = sdss_merged['umag_dr16'].combine_first(sdss_merged['umag_dr12'])
sdss_merged['gmag'] = sdss_merged['gmag_dr16'].combine_first(sdss_merged['gmag_dr12'])
sdss_merged['rmag'] = sdss_merged['rmag_dr16'].combine_first(sdss_merged['rmag_dr12'])
sdss_merged['imag'] = sdss_merged['imag_dr16'].combine_first(sdss_merged['imag_dr12'])
sdss_merged['zmag'] = sdss_merged['zmag_dr16'].combine_first(sdss_merged['zmag_dr12'])
sdss_merged['e_umag'] =
sdss_merged['e_umag_dr16'].combine_first(sdss_merged['e_umag_dr12'])
sdss_merged['e_gmag'] =
sdss_merged['e_gmag_dr16'].combine_first(sdss_merged['e_gmag_dr12'])
sdss_merged['e_rmag'] =
sdss_merged['e_rmag_dr16'].combine_first(sdss_merged['e_rmag_dr12'])
sdss_merged['e_imag'] =
sdss_merged['e_imag_dr16'].combine_first(sdss_merged['e_imag_dr12'])
sdss_merged['e_zmag'] =
sdss_merged['e_zmag_dr16'].combine_first(sdss_merged['e_zmag_dr12'])

# Drop the intermediate columns
sdss_merged = sdss_merged[['objID', 'RA_ICRS', 'DE_ICRS', 'zsp', 'umag', 'gmag', 'rmag',
'imag', 'zmag', 'e_umag', 'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']]

# Save the merged dataset for future use
sdss_merged.to_csv("sdss_merged.csv", index=False)
print(f"Merged SDSS dataset created with {len(sdss_merged)} rows.")

# Load the other datasets
twomass = pd.read_csv("2Mass.csv")  # II/246: 2MASS Extended Source Catalogue
gama = pd.read_csv("II356xmmom41s.csv")  # II/356: GAMA DR3

# Print columns for debugging
print("Columns in merged_data.csv:", df.columns.tolist())
print("Columns in sdss_merged (V/147 + V/154):", sdss_merged.columns.tolist())
print("Columns in twomass (II/246):", twomass.columns.tolist())
print("Columns in gama (II/356):", gama.columns.tolist())

# Function to clean DataFrame by removing rows with NaN or inf in RA/Dec columns
def clean_coordinates(df, ra_col, dec_col):
    initial_len = len(df)
```

```
    # Check for NaN or inf in RA and Dec columns
    mask = (
        df[ra_col].notna() & df[dec_col].notna() &  # Not NaN
        np.isfinite(df[ra_col]) & np.isfinite(df[dec_col])  # Not inf
    )
    df_cleaned = df[mask].copy()
    print(f"Removed {initial_len - len(df_cleaned)} rows from DataFrame due to NaN or inf in
{ra_col} or {dec_col}")
    # Print RA/Dec ranges for debugging
    if len(df_cleaned) > 0:
        print(f"RA range in {ra_col}: {df_cleaned[ra_col].min():.4f} to {df_cleaned[ra_col].max():.4f}")
        print(f"Dec range in {dec_col}: {df_cleaned[dec_col].min():.4f} to
{df_cleaned[dec_col].max():.4f}")
    else:
        print(f"No valid RA/Dec data after cleaning in {ra_col}/{dec_col}")
    return df_cleaned

# Cross-match function
def deg_to_rad(df, ra_col, dec_col):
    try:
        df['ra_rad'] = np.radians(df[ra_col])
        df['dec_rad'] = np.radians(df[dec_col])
        return df
    except KeyError as e:
        print(f"KeyError in deg_to_rad: {e}")
        print(f"Available columns in DataFrame: {df.columns.tolist()}")
        raise

def cross_match(df1, df2, ra_col1, dec_col1, ra_col2, dec_col2, max_dist_arcsec=10.0):
    try:
        # Clean DataFrames to remove rows with NaN or inf in RA/Dec
        df1_cleaned = clean_coordinates(df1, ra_col1, dec_col1)
        df2_cleaned = clean_coordinates(df2, ra_col2, dec_col2)

        # Check if either DataFrame is empty after cleaning
        if len(df1_cleaned) == 0 or len(df2_cleaned) == 0:
            print("One of the DataFrames is empty after cleaning. Cannot perform cross-match.")
            return df1_cleaned, pd.DataFrame()

        # Convert degrees to radians
        df1_cleaned = deg_to_rad(df1_cleaned, ra_col1, dec_col1)
        df2_cleaned = deg_to_rad(df2_cleaned, ra_col2, dec_col2)

        # Create coordinate arrays
```

```
        coords1 = np.array([df1_cleaned['ra_rad'], df1_cleaned['dec_rad']]).T
        coords2 = np.array([df2_cleaned['ra_rad'], df2_cleaned['dec_rad']]).T

        # Perform cross-match using cKDTree
        tree = cKDTree(coords2)
        max_dist = np.radians(max_dist_arcsec / 3600.0)
        dist, idx = tree.query(coords1, k=1, distance_upper_bound=max_dist)

        # Filter matches within the maximum distance
        matched = dist < max_dist
        df1_matched = df1_cleaned[matched].copy()
        df2_matched = df2_cleaned.iloc[idx[matched]].copy()

        # Reset indices
        df1_matched = df1_matched.reset_index(drop=True)
        df2_matched = df2_matched.reset_index(drop=True)

        # Print sample matches for debugging
        if len(df1_matched) > 0:
            print("Sample matches (first 5):")
            for i in range(min(5, len(df1_matched))):
                print(f"Match {i+1}: {ra_col1}={df1_matched[ra_col1].iloc[i]:.4f},
{dec_col1}={df1_matched[dec_col1].iloc[i]:.4f} "
                      f"matched to {ra_col2}={df2_matched[ra_col2].iloc[i]:.4f},
{dec_col2}={df2_matched[dec_col2].iloc[i]:.4f} "
                      f"(distance={(dist[matched][i] * 3600 * 180 / np.pi):.2f} arcsec)")
        else:
            print("No matches found within the specified radius.")

        return df1_matched, df2_matched
    except KeyError as e:
        print(f"KeyError in cross_match: {e}")
        print(f"df1 columns: {df1.columns.tolist()}")
        print(f"df2 columns: {df2.columns.tolist()}")
        raise
    except Exception as e:
        print(f"Error in cross_match: {e}")
        raise

# Store the original DataFrame before cross-matching
df_original = df.copy()

# Cross-match with VizieR catalogues
# SDSS merged dataset (V/147 + V/154, uses RA_ICRS, DE_ICRS)
```

```
df, sdss_matched = cross_match(df, sdss_merged, 'objra_y', 'objdec', 'RA_ICRS', 'DE_ICRS',
max_dist_arcsec=10.0)
print(f"After SDSS cross-match: {len(df)} galaxies")

# Check if cross-match succeeded; if not, fall back to original DataFrame
if len(df) == 0:
    print("No matches found with SDSS merged data (V/147 + V/154). Falling back to original
data and using nsa_z for redshift.")
    df = df_original.copy()
    df['zsp'] = df['nsa_z']
else:
    # Add zsp, umag, gmag, etc. from the merged SDSS dataset
    df = pd.concat([df, sdss_matched[['zsp', 'umag', 'gmag', 'rmag', 'imag', 'zmag', 'e_umag',
'e_gmag', 'e_rmag', 'e_imag', 'e_zmag']]], axis=1)

# II/246: 2MASS Extended Source Catalogue (uses RAJ2000, DEJ2000)
df, twomass_matched = cross_match(df, twomass, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000',
max_dist_arcsec=10.0)
print(f"After II/246 cross-match: {len(df)} galaxies")
if len(df) > 0:
    df = pd.concat([df, twomass_matched[['Jmag', 'Hmag', 'Kmag', 'e_Jmag', 'e_Hmag',
'e_Kmag']]], axis=1)

# II/356: GAMA DR3 (uses RAJ2000, DEJ2000)
df, gama_matched = cross_match(df, gama, 'objra_y', 'objdec', 'RAJ2000', 'DEJ2000',
max_dist_arcsec=10.0)
print(f"After II/356 cross-match: {len(df)} galaxies")
if len(df) > 0:
    df = pd.concat([df, gama_matched[['UmAB', 'BmAB', 'VmAB']]], axis=1)

# Recompute log_SFR_Ha and metallicity using fluxes from merged_data.csv
df['flux_Ha'] = df['F_Ha_cen']  # Use F_Ha_cen as flux_Ha
df['e_flux_Ha'] = df['e_F_Ha_cen']
df['flux_Hb'] = df['flux_Hbeta4861.36_Re_fit']
df['flux_OIII_5007'] = df['flux_[OIII]5006.84_Re_fit']
df['flux_NII_6584'] = df['flux_[NII]6583.45_Re_fit']

# Check for NaN in flux columns and handle them
flux_cols = ['flux_Ha', 'flux_Hb', 'flux_OIII_5007', 'flux_NII_6584', 'e_flux_Ha']
for col in flux_cols:
    nan_count = df[col].isna().sum()
    print(f"NaN count in {col}: {nan_count}")
    # Replace NaN with median for flux columns
    if nan_count > 0:
```

```
        df[col] = df[col].fillna(df[col].median())

df['Ha_Hb_observed'] = df['flux_Ha'] / df['flux_Hb']
Ha_Hb_intrinsic = 2.86
k_Ha = 2.468
k_Hb = 3.634
df['A_Ha'] = 2.5 * np.log10(df['Ha_Hb_observed'] / Ha_Hb_intrinsic) * (k_Ha / (k_Hb - k_Ha))
df['flux_Ha_corr'] = df['flux_Ha'] * 10**(0.4 * df['A_Ha'])
H0 = 70  # km/s/Mpc
c = 3e5  # km/s
df['DL'] = (c * df['zsp'] / H0) * 3.0856e24  # cm, using zsp
df['L_Ha'] = df['flux_Ha_corr'] * 4 * np.pi * df['DL']**2  # erg/s
df['log_SFR_Ha_raw'] = np.log10(df['L_Ha'] * 7.9e-42)  # M_sun/yr
df['log_O3N2_raw'] = np.log10((df['flux_OIII_5007'] / df['flux_Hb']) / (df['flux_NII_6584'] /
df['flux_Ha']))
df['OH_O3N2_raw'] = 8.533 - 0.214 * df['log_O3N2_raw']

# Add raw fluxes as features
df['log_flux_Ha'] = np.log10(df['flux_Ha'] + 1e-10)
df['log_flux_Hb'] = np.log10(df['flux_Hb'] + 1e-10)
df['log_flux_OIII_5007'] = np.log10(df['flux_OIII_5007'] + 1e-10)
df['log_flux_NII_6584'] = np.log10(df['flux_NII_6584'] + 1e-10)
df['log_e_flux_Ha'] = np.log10(df['e_flux_Ha'] + 1e-10)

# Add photometric colors (if available)
if 'umag' in df.columns:
    df['color_ug'] = df['umag'] - df['gmag']
    df['color_gr'] = df['gmag'] - df['rmag']
    df['color_ri'] = df['rmag'] - df['imag']
    df['color_iz'] = df['imag'] - df['zmag']
    df['e_color_ug'] = np.sqrt(df['e_umag']**2 + df['e_gmag']**2)
    df['e_color_gr'] = np.sqrt(df['e_gmag']**2 + df['e_rmag']**2)

# Add 2MASS colors (if available)
if 'Jmag' in df.columns:
    df['color_JK'] = df['Jmag'] - df['Kmag']
    df['e_color_JK'] = np.sqrt(df['e_Jmag']**2 + df['e_Kmag']**2)

# Drop rows with missing target or excessive NaNs
df = df.dropna(subset=["log_SFR_Ha"])
df = df.dropna(axis=0, thresh=int(0.8 * df.shape[1]))
df.fillna(df.median(numeric_only=True), inplace=True)

# Step 1: Cosmo-Rank Construction
```

```
gz_df = df.copy()
morph_cols = ['P(CD)', 'P(E)', 'P(S0)', 'P(Sa)', 'P(Sab)', 'P(Sb)', 'P(Sbc)',
              'P(Sc)', 'P(Scd)', 'P(Sd)', 'P(Sdm)', 'P(Sm)', 'P(Irr)']
gz_df['morph_sum'] = gz_df[morph_cols].sum(axis=1)
gz_df['Num_w'] = gz_df['conf_prob']
scaler_rank = MinMaxScaler()
gz_df[['morph_sum_norm', 'Num_w_norm', 'nsa_z_norm']] = scaler_rank.fit_transform(
    gz_df[['morph_sum', 'Num_w', 'nsa_z']]
)
gz_df['cosmo_rank'] = (0.5 * gz_df['Num_w_norm'] + 0.3 * gz_df['morph_sum_norm'] + 0.2 *
gz_df['nsa_z_norm'])
df['cosmo_rank'] = gz_df['cosmo_rank']

# Step 2: L_cosmo(s) Construction
alpha = -1.5
M_star = df['log_Mass_gas'].median()
df['a_n'] = (10 ** df['log_Mass_gas'])**(1 + alpha) * np.exp(-10 ** df['log_Mass_gas'] / (10 **
M_star))
bins = pd.qcut(df['Re_kpc'], q=20, labels=False, duplicates='drop') + 1
df['z_bin'] = bins
n_bins = df['z_bin'].nunique()
print(f"Number of unique bins after qcut: {n_bins}")
print("Bin distribution for Re_kpc (after qcut):")
print(df['z_bin'].value_counts().sort_index())

s_vals = [0.5, 1.0, 1.5, 2.0]
s_range = np.linspace(0.5, 2, 50)
for s in s_vals:
    col_name = f'L_cosmo_s_{s:.1f}'
    df[col_name] = df.groupby('z_bin')['a_n'].transform('mean') / (df['z_bin'] ** s)

print("Columns after creating L_cosmo_s*:")
print(df.columns.tolist())

print("Checking for NaN/infinite values in cosmo_rank and L_cosmo_s_1.0:")
print("cosmo_rank NaN count:", df['cosmo_rank'].isna().sum())
print("L_cosmo_s_1.0 NaN count:", df['L_cosmo_s_1.0'].isna().sum())
print("cosmo_rank infinite count:", np.isinf(df['cosmo_rank']).sum())
print("L_cosmo_s_1.0 infinite count:", np.isinf(df['L_cosmo_s_1.0']).sum())

df['cosmo_rank_L'] = df['cosmo_rank'] * df['L_cosmo_s_1.0']
df['cosmo_rank_L'] = df['cosmo_rank_L'].replace([np.inf, -np.inf], np.nan).fillna(0)
print("cosmo_rank_L created, NaN count:", df['cosmo_rank_L'].isna().sum())
```

```
# Plot L_cosmo(s) vs s
grouped_means = df.groupby('z_bin')['a_n'].mean().reindex(range(1, n_bins + 1), fill_value=0)
L_vals = [grouped_means / (np.arange(1, n_bins + 1) ** s) for s in s_range]
L_vals = np.array([sum(l) for l in L_vals])
plt.plot(s_range, L_vals)
plt.xlabel("s")
plt.ylabel("L_cosmo(s)")
plt.title("L_cosmo(s) Behavior")
plt.savefig("L_cosmo_curve.png")
plt.close()

# Scatter plot of L_cosmo_s_1.0 vs log_SFR_Ha
plt.scatter(df['L_cosmo_s_1.0'], df['log_SFR_Ha'], alpha=0.5)
plt.xlabel("L_cosmo_s_1.0")
plt.ylabel("log_SFR_Ha")
plt.title("L_cosmo_s_1.0 vs log_SFR_Ha")
plt.savefig("L_cosmo_s1_vs_SFR.png")
plt.close()

# Step 3: Define features and target
target = "log_SFR_Ha"
base_features = [
    "log_Mass_gas", "log_Mass", "Av_gas_Re", "Av_ssp_Re",
    "OH_O3N2_cen", "OH_T04_cen", "OH_dop_cen", "Age_LW_Re_fit",
    "ZH_LW_Re_fit", "Re_kpc", "vel_disp_Ha_cen", "Lambda_Re",
    "Sigma_Mass_Re", "EW_Ha_cen", "Ha_Hb_cen"
]
df["mass_metallicity"] = df["log_Mass"] * df["OH_O3N2_cen"]
df["dust_metallicity"] = df["Av_gas_Re"] * df["OH_T04_cen"]
df["disp_mass_ratio"] = df["vel_disp_Ha_cen"] / (df["Sigma_Mass_Re"] + 1e-5)
df["age_metallicity"] = df["Age_LW_Re_fit"] * df["ZH_LW_Re_fit"]
df["sqrt_Re_kpc"] = np.sqrt(df['Re_kpc'] + 1e-5)
df["log_mass"] = np.log(df["log_Mass_gas"] + 1e-5)
df["BSD_likelihood"] = (
    df["log_Mass_gas"] * df["OH_O3N2_cen"] /
    (df["Av_gas_Re"] + 1e-5 + df["Age_LW_Re_fit"])
)
df["cosmo_rank_mass"] = df["cosmo_rank"] * df["log_Mass"]
df["L_cosmo_s1_mass"] = df["L_cosmo_s_1.0"] * df["log_Mass"]
df["cosmo_rank_EW"] = df["cosmo_rank"] * df["EW_Ha_cen"]
df["L_cosmo_s1_EW"] = df["L_cosmo_s_1.0"] * df["EW_Ha_cen"]
df["cosmo_rank_scaled"] = df["cosmo_rank"] * 20
df["L_cosmo_s1_scaled"] = df["L_cosmo_s_1.0"] * 20
df["cosmo_rank_L_mass"] = df["cosmo_rank_L"] * df["log_Mass"]
```

```
df["L_cosmo_s1_metallicity"] = df["L_cosmo_s_1.0"] * df["OH_O3N2_cen"]
features = base_features + [
    "mass_metallicity", "dust_metallicity", "disp_mass_ratio",
    "age_metallicity", "sqrt_Re_kpc", "log_mass", "BSD_likelihood",
    "cosmo_rank", "L_cosmo_s_0.5", "L_cosmo_s_1.5", "L_cosmo_s_2.0",
    "cosmo_rank_L", "cosmo_rank_mass", "L_cosmo_s1_mass",
    "cosmo_rank_EW", "L_cosmo_s1_EW", "cosmo_rank_scaled", "L_cosmo_s1_scaled",
    "cosmo_rank_L_mass", "L_cosmo_s1_metallicity",
    "log_flux_Ha", "log_flux_Hb", "log_flux_OIII_5007", "log_flux_NII_6584",
    "log_e_flux_Ha", "log_SFR_Ha_raw", "OH_O3N2_raw"
]

# Add color features if available
if 'color_ug' in df.columns:
    features.extend(["color_ug", "color_gr", "color_ri", "color_iz", "e_color_ug", "e_color_gr"])
if 'color_JK' in df.columns:
    features.extend(["color_JK", "e_color_JK"])

X = df[features]
y = df[target]

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Normalize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Optuna optimization for Random Forest
def objective_rf(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 5, 20),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 10),
        'random_state': 42,
        'n_jobs': 2  # Use 2 CPU cores
    }
    model = RandomForestRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2', n_jobs=2)
    return scores.mean()

study_rf = optuna.create_study(direction='maximize')
```

```
study_rf.optimize(objective_rf, n_trials=20)
best_params_rf = study_rf.best_params
print("Best Random Forest Parameters:", best_params_rf)
print("Best Random Forest CV R²:", study_rf.best_value)

# Train Random Forest with best parameters
best_params_rf['n_jobs'] = 2  # Add n_jobs to the best parameters
rf = RandomForestRegressor(**best_params_rf, random_state=42)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
r2_rf = r2_score(y_test, y_pred_rf)

# Cross-validation for final RF model
cv_scores_rf = cross_val_score(rf, X_train_scaled, y_train, cv=5, scoring='r2', n_jobs=2)
print(f"Random Forest 5-Fold CV R²: Mean = {cv_scores_rf.mean():.4f}, Std =
{cv_scores_rf.std():.4f}")

# Optuna optimization for Gradient Boosting
def objective_gb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.2, log=True),
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'random_state': 42
    }
    model = GradientBoostingRegressor(**params)
    cv = KFold(n_splits=5, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='r2', n_jobs=2)
    return scores.mean()

study_gb = optuna.create_study(direction='maximize')
study_gb.optimize(objective_gb, n_trials=20)
best_params_gb = study_gb.best_params
print("Best Gradient Boosting Parameters:", best_params_gb)
print("Best Gradient Boosting CV R²:", study_gb.best_value)

# Train Gradient Boosting with best parameters
gb = GradientBoostingRegressor(**best_params_gb, random_state=42)
gb.fit(X_train_scaled, y_train)
y_pred_gb = gb.predict(X_test_scaled)
r2_gb = r2_score(y_test, y_pred_gb)

# Cross-validation for final GB model
cv_scores_gb = cross_val_score(gb, X_train_scaled, y_train, cv=5, scoring='r2', n_jobs=2)
```

```
print(f"Gradient Boosting 5-Fold CV R²: Mean = {cv_scores_gb.mean():.4f}, Std =
{cv_scores_gb.std():.4f}")

print(f"R^2 Score (Random Forest, Test Set): {r2_rf:.4f}")
print(f"R^2 Score (Gradient Boosting, Test Set): {r2_gb:.4f}")

# SHAP for feature selection
explainer_rf = shap.Explainer(rf, X_train_scaled)
shap_values_rf = explainer_rf(X_test_scaled)
explainer_gb = shap.Explainer(gb, X_train_scaled)
shap_values_gb = explainer_gb(X_test_scaled)
shap.summary_plot(shap_values_rf, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Random Forest")
plt.tight_layout()
plt.savefig("shap_rf_summary.png")
plt.clf()
shap.summary_plot(shap_values_gb, X_test_scaled, plot_type="bar", show=False)
plt.title("SHAP Summary - Gradient Boosting")
plt.tight_layout()
plt.savefig("shap_gb_summary.png")
plt.clf()

# Feature selection based on SHAP
shap_values_rf_mean = np.abs(shap_values_rf.values).mean(axis=0)
feature_importance = pd.DataFrame({"feature": features, "importance": shap_values_rf_mean})
feature_importance = feature_importance.sort_values("importance", ascending=False)
top_features = feature_importance["feature"].head(10).tolist()
bsd_features = ["cosmo_rank", "L_cosmo_s_1.0", "cosmo_rank_L", "cosmo_rank_scaled",
"L_cosmo_s1_scaled", "cosmo_rank_L_mass", "L_cosmo_s1_metallicity"]
for bsd_feature in bsd_features:
    if bsd_feature not in top_features:
        top_features.append(bsd_feature)
X_selected = df[top_features]
X_train_selected, X_test_selected, _, _ = train_test_split(X_selected, y, test_size=0.2,
random_state=42)
X_train_selected_scaled = scaler.fit_transform(X_train_selected)
X_test_selected_scaled = scaler.transform(X_test_selected)

# Symbolic Regression with gplearn
symbolic_model = SymbolicRegressor(
    population_size=3000,
    generations=150,
    stopping_criteria=0.01,
    p_crossover=0.7,
```

```
    p_subtree_mutation=0.1,
    p_hoist_mutation=0.05,
    p_point_mutation=0.1,
    max_samples=0.9,
    verbose=1,
    parsimony_coefficient=0.0001,
    random_state=42,
    function_set=('add', 'sub', 'mul', 'div', 'sqrt', 'log', 'sin', 'cos'),
    n_jobs=2  # Use 2 CPU cores
)
symbolic_model.fit(X_train_selected_scaled, y_train)
y_pred_sym = symbolic_model.predict(X_test_selected_scaled)
r2_sym = r2_score(y_test, y_pred_sym)
print(f"gplearn R²: {r2_sym:.4f}")
print("Symbolic Expression:")
print(symbolic_model._program)

# Cross-validation for symbolic model
cv_scores_sym = cross_val_score(symbolic_model, X_train_selected_scaled, y_train, cv=5,
scoring='r2', n_jobs=2)
print(f"Symbolic Regression 5-Fold CV R²: Mean = {cv_scores_sym.mean():.4f}, Std =
{cv_scores_sym.std():.4f}")

# Save symbolic expression as polynomial fit plot and print the equation
x = np.linspace(min(y_test), max(y_test), 500)
y_expr = symbolic_model.predict(scaler.transform(np.tile(X_test_selected.mean().values,
(500,1))))
p = Polynomial.fit(y_test, y_pred_sym, deg=3)
# Extract polynomial coefficients
coeffs = p.coef
print("Polynomial Fit Equation (degree 3):")
print(f"y = {coeffs[0]:.4f} + {coeffs[1]:.4f}*x + {coeffs[2]:.4f}*x^2 + {coeffs[3]:.4f}*x^3")
# Plot the polynomial fit
plt.plot(*p.linspace(), label="Polynomial Fit")
plt.scatter(y_test, y_pred_sym, s=10, alpha=0.5, label="Symbolic Predictions")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted log_SFR_Ha")
plt.title("Symbolic Regression Fit with Polynomial")
plt.legend()
plt.tight_layout()
plt.savefig("gplearn_expression_plot.png")
plt.clf()

# Plot predictions vs true
```

```
plt.scatter(y_test, y_pred_rf, alpha=0.5, label="RF", marker="o")
plt.scatter(y_test, y_pred_gb, alpha=0.5, label="GB", marker="s")
plt.scatter(y_test, y_pred_sym, alpha=0.5, label="Symbolic", marker="^")
plt.plot([y.min(), y.max()], [y.min(), y.max()], "k--")
plt.xlabel("True log_SFR_Ha")
plt.ylabel("Predicted")
plt.title("Model Predictions vs True Values")
plt.legend()
plt.tight_layout()
plt.savefig("model_comparison.png")
plt.clf()

# Residuals and Correlation Analysis
residuals = pd.DataFrame({
    "True": y_test,
    "RF": y_pred_rf - y_test,
    "GB": y_pred_gb - y_test,
    "Symbolic": y_pred_sym - y_test,
    "cosmo_rank": X_test["cosmo_rank"],
    "L_cosmo_s1": X_test["L_cosmo_s_1.0"]
})
sns.scatterplot(data=residuals, x="True", y="Symbolic", hue="cosmo_rank",
size="L_cosmo_s1", alpha=0.6)
plt.axhline(0, color="black", linestyle="--")
plt.title("Symbolic Residuals vs True log_SFR_Ha")
plt.savefig("residuals_sym.png")
plt.clf()

print("Correlation of BSD Features with Residuals:")
print(residuals[["RF", "GB", "Symbolic", "cosmo_rank", "L_cosmo_s1"]].corr()[["RF", "GB",
"Symbolic"]])
```